

CLOUD-NATIVE DEVOPS, DEVSECOPS & SRE

12-DAY PRODUCTION ENGINEERING PROGRAM

24 September - 10 October 2026

72 Hours Instruction + Holiday GitHub Mini-Projects + 6-Hour Final Production Challenge

PYTHON BACKEND	GIT + LINUX	DOCKER + CI/CD	TERRAFORM	KUBERNETES + GITOPS	OBSERVABILITY + DEVSECOPS	SRE + PRODUCTION
-------------------	----------------	-------------------	-----------	------------------------	------------------------------	---------------------

Designed for students who already know Python backend development and have prior AWS/Azure exposure.

Program Design

One continuous application. Repeated productionization. Evidence through GitHub.

Instructional design: approximately 30% concept/demo and 70% guided practice, independent implementation, troubleshooting and GitHub evidence.

Prerequisites

- Python backend fundamentals: REST APIs, packages, virtual environments and basic testing.
- Prior AWS/Azure learning or certification. Cloud fundamentals are not retaught in this program.
- Basic command-line familiarity and a GitHub account.

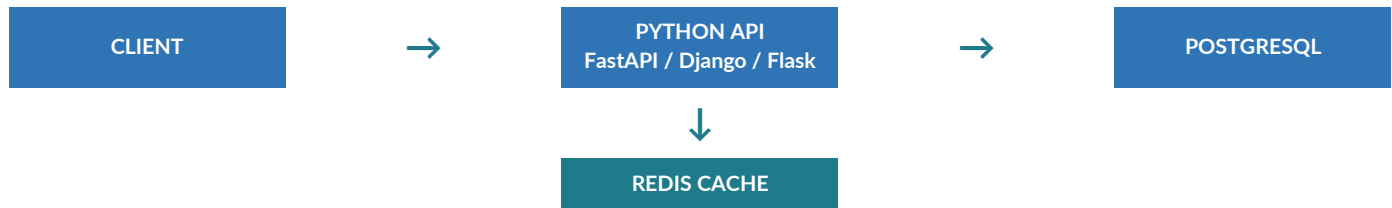
Program outcome

By the end of the program, each student should be able to take a Python backend repository and: test it, containerize it, build CI/CD, provision infrastructure through Terraform, deploy to Kubernetes, package with Helm, operate deployment through GitOps, add observability and security gates, scale it, troubleshoot it and recover from failures.

Core toolchain

Foundation	Build & Delivery	Infrastructure	Platform	Operations
Linux Git/GitHub	Docker GitHub Actions Registry	Terraform AWS or Azure	Kubernetes Helm Argo CD	Prometheus Grafana OpenTelemetry basics Trivy SRE

Common application used throughout the program



The same repository evolves from local development to a monitored, secured and recoverable production-style deployment. Students do not build 12 unrelated lab applications.

Program Calendar

12 instructional days + 5 holiday project days + final production evaluation

Stage	Date	Focus	Mode
Day 1	Thu, Sep 24	Linux + Git/GitHub for DevOps	Class - 6 h
Day 2	Fri, Sep 25	Docker Fundamentals	Class - 6 h
Day 3	Mon, Sep 28	Docker Compose + Multi-Service App	Class - 6 h
Holiday	Sat-Sun, Sep 26-27	Mini Project 1: Containerized Production Backend	GitHub task
Day 4	Tue, Sep 29	Continuous Integration - GitHub Actions	Class - 6 h
Day 5	Wed, Sep 30	Registry + Continuous Delivery	Class - 6 h
Day 6	Thu, Oct 1	Terraform - Infrastructure as Code	Class - 6 h
Day 7	Mon, Oct 5	Terraform + Complete Cloud CI/CD	Class - 6 h
Holiday	Fri-Sun, Oct 2-4	Mini Project 2: Infrastructure + Automated Deployment	GitHub task
Day 8	Tue, Oct 6	Kubernetes Fundamentals	Class - 6 h
Day 9	Wed, Oct 7	Production Kubernetes	Class - 6 h
Day 10	Thu, Oct 8	Helm + GitOps + Argo CD	Class - 6 h
Day 11	Fri, Oct 9	Observability + SRE	Class - 6 h
Day 12	Sat, Oct 10	DevSecOps + Production Integration	Class - 6 h
Evaluation	Sat, Oct 10	Final Production Reliability Challenge	6 h assessment

Standard 6-hour classroom rhythm

Time	Mode	Purpose
00:00-00:30	Review + troubleshooting	Previous-day issues, short viva/quiz, correction of misconceptions
00:30-01:30	Concept + architecture	Why the technology exists, core terminology, production use
01:30-02:30	Trainer live implementation	Build once end-to-end with explanation
02:30-04:30	Guided student lab	Students reproduce and modify the implementation
04:30-05:30	Independent challenge	Failure scenario, debugging or extension without step-by-step help
05:30-06:00	GitHub evidence + viva	Commit/PR/release evidence and trainer verification

DAY 1

Thursday, 24 September 2026
Linux + Git/GitHub for DevOps

Learning outcomes

Use Linux confidently for application operations • Work through a professional Git branch/PR workflow • Diagnose basic process, permission and networking issues

Syllabus coverage

- Linux filesystem, paths, files, permissions, users/groups and sudo
- Processes, services, environment variables, pipes, grep, logs, curl and basic networking commands
- SSH fundamentals and secure remote access concepts
- Git: clone, status, add, commit, diff, log, branches, merge and tags
- GitHub: issues, branches, pull requests, reviews, merge conflicts and .gitignore
- Repository hygiene: README, environment files, commit messages and issue linking

6-hour classroom plan

Time	Activity	Expected output
0:00-0:30	Baseline check	Verify GitHub, Git, Python and terminal readiness
0:30-1:30	Linux operations	Filesystem, permissions, processes, networking and logs
1:30-2:30	Git/GitHub demo	Issue → branch → commit → PR → review → merge
2:30-4:30	Guided lab	Clone starter API; inspect logs; modify configuration; complete PR workflow
4:30-5:30	Failure challenge	Fix permission/process/merge-conflict scenario
5:30-6:00	GitHub evidence + viva	Merged PR + clean history + short command viva

After-class activity

- Add or improve `/health` endpoint.
- Improve README with setup instructions.
- Open one GitHub issue, solve it on a branch and merge through a PR.

Mandatory GitHub evidence

- At least 3 meaningful commits
- One issue and one merged PR
- Updated README and health endpoint

Trainer verification

- Student can explain branch vs commit vs PR.
- Student can find a running process and inspect logs.
- No secrets or local virtual-environment files committed.

DAY 2**Friday, 25 September 2026**
Docker Fundamentals**Learning outcomes**

Build and run container images • Explain image layers and container lifecycle • Containerize the existing Python backend securely

Syllabus coverage

- Containers vs virtual machines; images, containers and registries
- Dockerfile instructions: FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, CMD and ENTRYPOINT
- Build, run, exec, logs, inspect, stop, remove and prune
- Ports, volumes, bind mounts and Docker networks
- Environment variables and `.dockerignore`
- Layer caching, image size, multi-stage build concept and non-root execution

6-hour classroom plan

Time	Activity	Expected output
0:00-0:30	Review	Git/Linux correction and repo readiness
0:30-1:30	Docker concepts	Images, layers, lifecycle, registry and networking
1:30-2:30	Live build	Write Dockerfile and containerize Python API
2:30-4:30	Guided lab	Build/run/debug image; ports; env vars; volumes; inspect layers
4:30-5:30	Optimization challenge	Reduce context/layers and run as non-root where feasible
5:30-6:00	GitHub evidence + viva	Dockerfile commit + successful local run

After-class activity

- Optimize Dockerfile and `.dockerignore`.
- Document build/run/debug commands in README.
- Capture image size before/after optimization.

Mandatory GitHub evidence

- Dockerfile + `.dockerignore`
- README Docker section
- Evidence of successful container health endpoint

Trainer verification

- Student can rebuild from a clean checkout.
- No host-specific absolute paths.
- Container starts using documented command.

DAY 3**Monday, 28 September 2026****Docker Compose + Multi-Service Application****Learning outcomes**

Run a multi-container backend stack • Persist data and manage service communication • Prepare directly for Holiday Mini Project 1

Syllabus coverage

- Docker Compose structure and YAML basics
- Services, networks, named volumes, environment files and restart policies
- PostgreSQL container configuration and data persistence
- Redis caching integration
- Service DNS, ports and dependency concepts
- Health checks and startup ordering considerations
- Operational commands: up, down, logs, ps, exec and rebuild

6-hour classroom plan

Time	Activity	Expected output
0:00-0:30	Review	Dockerfile quality and common image problems
0:30-1:30	Compose architecture	API + PostgreSQL + Redis design
1:30-2:30	Live implementation	Build complete compose stack
2:30-4:30	Guided lab	Students run, test persistence, cache and inter-service networking
4:30-5:30	Failure challenge	Break DB/Redis/config; diagnose with logs/inspect
5:30-6:00	Holiday briefing	Create project issues, milestones and acceptance checklist

After-class activity

- Create GitHub issues for Mini Project 1 before leaving class.
- Freeze a clean starter tag for the holiday task.
- Do not finish the holiday project in class; only confirm all required concepts work.

Mandatory GitHub evidence

- `docker-compose.yml`
- `.env.example` without secrets
- Working API + PostgreSQL + Redis locally
- Holiday project issue board/checklist

Trainer verification

- All students can start stack from README.
- Persistence survives container recreation.
- Trainer confirms project requirements before holiday.

HOLIDAY MINI PROJECT 1

Saturday-Sunday, 26-27 September 2026

Purpose: independently apply Days 1-3. This is the first proof that a student can move from source code to a reproducible multi-container development environment without trainer hand-holding.

26 September - Build the containerized backend

- Create/complete a Python backend with at least four meaningful API endpoints.
- Use PostgreSQL for persistent data and Redis for caching or a clearly documented supporting use case.
- Create Dockerfile and Docker Compose configuration for the complete stack.
- Use environment variables; commit `.env.example`, never actual credentials.
- Implement `/health` endpoint and service/container health checks where appropriate.
- Verify container-to-container networking and database persistence.

27 September - Productionize and document

- Optimize Dockerfile and build context; use `.dockerignore`.
- Use a non-root runtime user where technically feasible.
- Test stop/start/recreate behavior and verify data persistence.
- Create architecture diagram using Mermaid or equivalent repository-friendly format.
- Write complete README: prerequisites, start/stop, testing, troubleshooting and cleanup.
- Use issues/branches/PRs and meaningful commit history; finish with a release tag.

Mandatory deliverables

- GitHub repository - no ZIP submission
- Working Python + PostgreSQL + Redis stack
- Dockerfile, Compose file, `.env.example`, `.gitignore` and README
- Tests or test evidence for core APIs
- Architecture diagram and troubleshooting notes
- Release tag `holiday-project-1-v1.0`

Assessment rubric

Criterion	Weight
Functionality and reproducibility	35%
Docker/Compose quality	25%
Git/GitHub workflow	15%
Documentation and architecture	15%
Troubleshooting/viva evidence	10%

Required release tag: holiday-project-1-v1.0

- Individual submission is preferred so every student proves competence.
- Trainer should sample-clone repositories and run them from README instructions.
- A working app with poor Git history or hard-coded secrets must not receive full marks.

DAY 4**Tuesday, 29 September 2026**
Continuous Integration - GitHub Actions**Learning outcomes**

Automate code quality and test checks on push/PR • Read CI logs and diagnose failed jobs • Protect the main branch using automated verification

Syllabus coverage

- CI principles, events/triggers, runners, jobs, steps and actions
- GitHub Actions YAML structure
- Python dependency installation and caching
- Linting, unit tests and API tests
- Artifacts and test output
- Docker image build as a CI step
- Branch protection/PR checks concept and workflow status badges

6-hour classroom plan

Time	Activity	Expected output
0:00-0:45	Mini Project 1 review	Select submissions; identify Docker/Compose quality issues
0:45-1:30	CI concepts	Pipeline stages, runner and failure visibility
1:30-2:30	Live workflow	Checkout → install → lint → test → Docker build
2:30-4:15	Guided lab	Students add CI to own holiday project
4:15-5:15	Break-it challenge	Trainer/student intentionally causes lint/test/build failure and diagnoses it
5:15-6:00	GitHub controls	PR check, badge, evidence and viva

After-class activity

- Intentionally break one CI stage, capture failure reason, then fix it.
- Add CI status badge to README.
- Ensure CI runs on pull requests.

Mandatory GitHub evidence

- ``.github/workflows/ci.yml``
- Successful workflow run
- One documented failed-and-fixed CI run

Trainer verification

- Workflow is reproducible from clean push.
- Student explains which stage failed and why.
- No secrets printed into logs.

DAY 5**Wednesday, 30 September 2026**
Container Registry + Continuous Delivery**Learning outcomes**

Publish versioned images automatically • Separate development/staging/production delivery concepts • Perform and document a safe rollback

Syllabus coverage

- CI vs CD and deployment lifecycle
- Container registry workflow: GitHub Container Registry or selected cloud registry
- Tagging: latest pitfalls, semantic versions and commit SHA tags
- GitHub Secrets and environment-scoped variables
- Deployment to a prepared cloud VM
- Release notes and artifact traceability
- Rollback strategies and basic zero/low-downtime concepts

6-hour classroom plan

Time	Activity	Expected output
0:00-0:30	Review	CI failures and workflow hygiene
0:30-1:30	CD concepts	Artifact promotion, tags, environments and rollback
1:30-2:30	Live pipeline	Build → tag → push → deploy
2:30-4:30	Guided lab	Students publish images and deploy to prepared VM
4:30-5:30	Rollback challenge	Deploy bad version then recover using previous image
5:30-6:00	Release evidence	Create release/tag and verify traceability

After-class activity

- Create one semantic release tag.
- Write a short rollback runbook.
- Record which commit SHA produced the deployed image.

Mandatory GitHub evidence

- Registry image with versioned tag
- CD workflow or deployment script
- `docs/rollback.md`
- GitHub release/tag

Trainer verification

- No manual SCP of application source files.
- Student can identify deployed image version.
- Rollback steps are tested, not theoretical.

DAY 6**Thursday, 1 October 2026****Terraform - Infrastructure as Code****Learning outcomes**

Provision cloud infrastructure from code • Understand Terraform state and lifecycle • Replace console-click provisioning with repeatable configuration

Syllabus coverage

- Infrastructure as Code principles and declarative configuration
- HCL syntax, providers, resources, variables, outputs and data sources
- `terraform init`, `fmt`, `validate`, `plan`, `apply` and `destroy`
- State purpose, state sensitivity and remote-state concept
- Network, subnet, security rules and compute resources
- `.tfvars`, environment separation and modules introduction
- Drift concept and idempotent infrastructure changes

6-hour classroom plan

Time	Activity	Expected output
0:00-0:30	Review	Delivery pipeline and cloud target readiness
0:30-1:30	Terraform model	Configuration, provider, plan, state and lifecycle
1:30-2:30	Live infrastructure	Provision network/security/VM
2:30-4:30	Guided lab	Students parameterize and deploy infrastructure
4:30-5:30	Change challenge	Modify infrastructure safely; interpret plan before apply
5:30-6:00	Cleanup + evidence	Outputs, validation, cost-conscious destroy where appropriate

After-class activity

- Move hard-coded values to variables.
- Add meaningful outputs.
- Run `fmt` and `validate`; capture clean result.

Mandatory GitHub evidence

- `terraform/main.tf`
- `terraform/variables.tf`
- `terraform/outputs.tf`
- `.gitignore` covering Terraform-sensitive/generated files

Trainer verification

- No cloud credentials committed.
- Student explains state and plan.
- Infrastructure can be recreated from repository code.

DAY 7

Monday, 5 October 2026
Terraform + Complete Cloud CI/CD

Learning outcomes

Connect application delivery to infrastructure automation • Prepare all knowledge required for Holiday Mini Project 2 •
 Demonstrate one end-to-end production-style flow

Syllabus coverage

- Terraform in a delivery workflow
- Infrastructure and application pipeline boundaries
- Environment variables, secrets and protected deployment environments
- Registry image promotion and cloud deployment
- Health verification after deployment
- Versioning and rollback using immutable image tags
- Operational evidence and cloud-resource cleanup

6-hour classroom plan

Time	Activity	Expected output
0:00-0:30	Review	Terraform mistakes and security checks
0:30-1:15	Architecture	End-to-end Git → CI → image → registry → IaC → deploy
1:15-2:30	Trainer demonstration	Complete flow from commit to running cloud application
2:30-4:15	Guided integration	Students connect their CI/CD and Terraform components
4:15-5:00	Recovery drill	Bad release / health failure / rollback
5:00-6:00	Holiday Project 2 briefing	Issues, milestones, acceptance criteria and cost controls

After-class activity

- Create detailed GitHub issues for Oct 2-4 tasks.
- Confirm Terraform and pipeline can run independently before leaving class.
- Create a pre-holiday baseline tag.

Mandatory GitHub evidence

- Working end-to-end proof on selected cloud
- Holiday project board/checklist
- Baseline release tag

Trainer verification

- Every student understands the project acceptance criteria.
- Trainer verifies no credentials in repository.
- Students know how and when to destroy chargeable resources.

HOLIDAY MINI PROJECT 2

Friday-Sunday, 2-4 October 2026

Purpose: integrate CI/CD, container registry and Terraform into one reproducible cloud deployment. Day 7 is deliberately planned as the end-to-end demonstration immediately before this project.

2 October - Infrastructure as Code

- Create Terraform configuration for the chosen common cloud: network/subnet, security rules and compute instance.
- Use variables and outputs; do not hard-code credentials or student-specific absolute paths.
- Run `terraform fmt`, `validate` and `plan` successfully.
- Document state handling and resource cleanup expectations.
- Capture only non-sensitive proof of created resources.

3 October - CI/CD automation

- On push/merge, run lint/tests before any deployment stage.
- Build a Docker image and tag it with a version and/or commit SHA.
- Push the image to the selected container registry.
- Deploy using the produced image; do not copy source code manually to the server.
- Use repository/environment secrets correctly and avoid secret leakage in logs.

4 October - Reliability, rollback and documentation

- Deploy a new version and verify application health.
- Simulate or introduce a controlled bad release, detect the failure and roll back.
- Document architecture, CI/CD flow, deployment steps and troubleshooting.
- Tag the completed release and preserve workflow evidence.
- Destroy chargeable resources that are no longer needed after evidence is captured.

Mandatory deliverables

- Terraform configuration under `terraform/`
- CI workflow with lint/test/build stages
- Versioned image in registry
- Automated or scripted deployment evidence
- Successful health verification and tested rollback
- Architecture + CI/CD diagram + troubleshooting notes
- Release tag `holiday-project-2-v2.0`

Assessment rubric

Criterion	Weight
Infrastructure as Code	25%
CI quality	20%
CD and registry traceability	20%
Rollback/reliability	15%
Security and secrets hygiene	10%
Documentation/viva	10%

Required release tag: holiday-project-2-v2.0

- The common classroom cloud should be standardized to one platform for consistency and trainer support.
- Students may reproduce the same architecture on a second cloud later, but dual-cloud teaching is not part of these 72 hours.
- Cost control is part of professional practice: unnecessary resources must be destroyed after verification.

DAY 8

Tuesday, 6 October 2026
Kubernetes Fundamentals

Learning outcomes

Understand Kubernetes desired-state model • Deploy container images using Kubernetes manifests • Scale and expose an application locally

Syllabus coverage

- Cluster, control plane and worker-node concepts
- Pods, ReplicaSets and Deployments
- Namespaces, labels, selectors and annotations
- Declarative YAML manifests and `kubectl` workflow
- Services: ClusterIP and exposure concepts
- Scaling replicas and rollout basics
- Local learning environment: kind or minikube

6-hour classroom plan

Time	Activity	Expected output
0:00-0:45	Mini Project 2 review	Inspect IaC/CD submissions and discuss failure patterns
0:45-1:30	Kubernetes model	Desired state, pod lifecycle and controllers
1:30-2:30	Live deployment	Deploy existing registry image to local cluster
2:30-4:30	Guided lab	Deployment, Service, scaling, logs, exec and delete/reconcile
4:30-5:30	Failure challenge	Delete pod/change image/misconfigure selector and diagnose
5:30-6:00	GitHub evidence	Commit clean `k8s/` manifests

After-class activity

- Create namespace-aware manifests.
- Scale application and record observed behavior.
- Document five essential `kubectl` troubleshooting commands.

Mandatory GitHub evidence

- `k8s/deployment.yaml`
- `k8s/service.yaml`
- README Kubernetes run section

Trainer verification

- Student explains Pod vs Deployment vs Service.
- Manifests recreate deployment from clean cluster.
- No use of manually created objects that are absent from Git.

DAY 9

Wednesday, 7 October 2026
Production Kubernetes

Learning outcomes

Harden Kubernetes deployment for reliability • Use probes, resources, configuration and rollback • Troubleshoot common Kubernetes failures

Syllabus coverage

- ConfigMaps and Secrets
- Resource requests and limits
- Liveness, readiness and startup probes
- Ingress or Gateway concept and service routing
- Persistent storage concepts
- Horizontal Pod Autoscaler concept and exercise
- Rollout status/history, image update and rollback
- Troubleshooting with logs, describe and events

6-hour classroom plan

Time	Activity	Expected output
0:00-0:30	Review	Manifest hygiene and controller behavior
0:30-1:30	Production configuration	Probes, resources, ConfigMaps, Secrets and routing
1:30-2:30	Live hardening	Add probes/resources/config to deployment
2:30-4:15	Guided lab	Full app deploy with failure-safe configuration
4:15-5:15	Incident drill	Failed readiness / crash / bad image / misconfig; diagnose and recover
5:15-6:00	Autoscale + rollback	Load exercise, rollout history and recovery

After-class activity

- Write one Kubernetes incident note: symptom → command used → root cause → fix.
- Clean secrets/config handling.
- Validate rollback procedure.

Mandatory GitHub evidence

- ConfigMap/Secret manifests or secure secret approach
- Probe and resource configuration
- Incident note under `docs/`

Trainer verification

- Student can explain readiness vs liveness.
- Trainer sees actual rollback/recovery.
- Application receives traffic only when ready.

DAY 10**Thursday, 8 October 2026****Helm + GitOps + Argo CD****Learning outcomes**

Package Kubernetes configuration as a reusable Helm chart • Understand Git as deployment source of truth • Deploy and reconcile application through GitOps

Syllabus coverage

- Why Helm; Chart.yaml, values.yaml and templates
- Template values, environment overrides, lint/install/upgrade/rollback
- GitOps principles: declarative, versioned and reconciled state
- Argo CD applications, sync, health and drift
- Separation of application source and deployment configuration concepts
- Dev/prod values and controlled promotion

6-hour classroom plan

Time	Activity	Expected output
0:00-0:30	Review	Kubernetes incident learnings
0:30-1:30	Helm	Chart structure, templates and values
1:30-2:30	Live conversion	Convert existing manifests into Helm chart
2:30-3:45	Guided Helm lab	Lint/install/upgrade/rollback
3:45-4:45	Argo CD/GitOps demo	Git change → sync → cluster state
4:45-5:30	Drift challenge	Manual cluster change then observe reconciliation
5:30-6:00	GitHub evidence	Commit chart + environment values + GitOps notes

After-class activity

- Create `values-dev.yaml` and `values-prod.yaml`.
- Document what happens when cluster state differs from Git.
- Perform one Helm rollback or Argo CD resync.

Mandatory GitHub evidence

- `helm/Chart.yaml` + templates
- Environment values files
- GitOps deployment evidence

Trainer verification

- Chart passes lint.
- Student can trace a deployment change to a Git commit.
- Manual cluster edits are not treated as the source of truth.

DAY 11**Friday, 9 October 2026****Observability + Site Reliability Engineering****Learning outcomes**

Observe application health using metrics/logs/traces concepts • Build actionable dashboards and alerts • Use SRE language to reason about reliability and incidents

Syllabus coverage

- Monitoring vs observability
- Metrics, logs and traces; OpenTelemetry concepts
- Prometheus targets, exporters and basic PromQL
- Grafana dashboards and meaningful panels
- Application metrics: request count, latency, errors and saturation concepts
- SLI, SLO, SLA and error-budget concepts
- Incident response, MTTR and blameless postmortem structure

6-hour classroom plan

Time	Activity	Expected output
0:00-0:30	Review	GitOps behavior and deployment traceability
0:30-1:30	Observability concepts	Signals, instrumentation and SRE measures
1:30-2:30	Live stack	Prometheus + Grafana + application metrics
2:30-4:15	Guided lab	Create dashboard; generate load; observe behavior
4:15-5:15	Incident simulation	Cause failure/high latency and investigate using telemetry
5:15-6:00	SRE closeout	Define simple SLI/SLO and draft incident summary

After-class activity

- Export Grafana dashboard JSON.
- Write a one-page incident/postmortem.
- Define one availability or latency SLI and corresponding SLO.

Mandatory GitHub evidence

- `monitoring/` configuration
- Grafana dashboard export
- `docs/incident-report.md`
- SLO statement

Trainer verification

- Dashboard shows useful operational signals, not decorative panels.
- Student connects symptom to metric/log evidence.
- Postmortem includes root cause, impact, recovery and prevention.

DAY 12

Saturday, 10 October 2026
DevSecOps + Production Integration

Learning outcomes

Add automated security checks to the software delivery lifecycle • Remediate findings instead of only generating reports • Integrate all program components before final evaluation

Syllabus coverage

- Shift-left security and threat-aware delivery
- Dependency and Python package scanning concepts
- Container image vulnerability scanning
- Secret scanning and repository hygiene
- IaC and Kubernetes misconfiguration scanning
- Trivy workflow; SAST/SonarQube concept where available
- Kubernetes security context, least privilege and non-root containers
- SBOM concept and security gates in CI/CD
- Final pipeline integration and runbook preparation

6-hour classroom plan

Time	Activity	Expected output
0:00-0:30	Review	SRE incident learnings
0:30-1:30	DevSecOps concepts	Threats, scanners, severity and gating
1:30-2:30	Live security pipeline	Scan repo/image/IaC/Kubernetes configuration
2:30-4:00	Guided remediation	Fix selected findings and re-run pipeline
4:00-5:00	Production integration	CI + security + image + Helm/GitOps + observability check
5:00-6:00	Final readiness review	README, architecture, runbook, clean repo and Oct 10 briefing

After-class activity

- Fix high-confidence/high-severity findings identified in the lab.
- Complete production runbook and architecture documentation.
- Ensure final repository can be evaluated from README alone.

Mandatory GitHub evidence

- Security workflow/job
- Scan reports or summarized evidence
- `docs/runbook.md`
- Final architecture + deployment documentation

Trainer verification

- Secrets are absent from Git history and logs.
- Student explains at least one remediated finding.
- All major folders and workflows are committed before final challenge.

Final Production Reliability Challenge

Saturday, 10 October 2026 | 6-hour practical evaluation - not counted inside the 72 instructional hours

This is not another teaching day and should not be replaced by an MCQ exam. Students must demonstrate deployment, observability, security and recovery under time pressure.

Challenge brief

Example change request: Add a new API capability and release the next production version without disrupting the currently working service. The release must pass CI and security checks, deploy through the configured delivery/GitOps path, become observable, and support recovery if the new version is unhealthy.

Expected end-to-end path

CODE	GIT	CI	SECURITY	IMAGE	REGISTRY	HELM / ARGO CD	K8S	OBSERVE / RECOVER
------	-----	----	----------	-------	----------	-------------------	-----	----------------------

6-hour evaluation flow

Time	Student activity	Evidence
0:00-0:30	Receive change request, create issue/branch and make implementation plan	Issue + branch
0:30-1:30	Implement code change and tests	Commits + passing local tests
1:30-2:30	Run CI, security scans, build/tag/publish image	Successful pipeline + immutable image tag
2:30-3:30	Deploy through Helm/GitOps and verify readiness	Deployment evidence + health check
3:30-4:30	Trainer injects one controlled production failure	Observed symptom and diagnostic trail
4:30-5:30	Diagnose, fix or roll back, verify recovery	Recovery evidence + monitoring signals
5:30-6:00	Write incident summary, final PR/release and viva	Incident note + release tag + viva

Possible trainer-injected failures

- Wrong environment variable or secret reference
- Bad image tag or image-pull failure
- Failed readiness/liveness configuration
- Application crash or dependency connectivity failure
- Incorrect Kubernetes selector/service configuration
- Resource pressure or misconfigured requests/limits
- Broken deployment value requiring rollback

Final challenge scoring

Area	Marks
Git workflow + implementation quality	15
CI/CD + artifact traceability	20
Kubernetes/Helm/GitOps deployment	20
DevSecOps gates	10
Observability + diagnosis	15
Recovery/rollback	15
Incident report + viva	5

GitHub Portfolio Standard

Every class day must leave visible engineering evidence

Recommended final repository structure

```
studentname-cloud-native-devops/
├── app/
├── tests/
├── Dockerfile
├── docker-compose.yml
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   └── outputs.tf
├── k8s/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── configmap.yaml
├── helm/
│   ├── Chart.yaml
│   ├── values.yaml
│   └── templates/
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── cd.yml
├── monitoring/
├── security/
├── docs/
│   ├── architecture.md
│   ├── deployment.md
│   ├── rollback.md
│   ├── runbook.md
│   └── incident-report.md
├── .env.example
├── .gitignore
└── README.md
```

Mandatory workflow discipline

- Daily pattern: Issue → Branch → Work → Commit → Pull Request → Review → Merge.
- No complete project upload as one final-day commit. Commit history is part of assessment evidence.
- No passwords, API keys, `.env` files, cloud credentials or private keys in the repository or workflow logs.
- Every important deployment must be traceable to a commit SHA and an image/release tag.
- README must let a reviewer understand architecture, prerequisites, local run, deployment, validation, rollback and cleanup.
- Generated scanner output can be summarized if raw reports are noisy, but remediated findings must be explained.

Program Assessment & Governance

Simple enough for trainers to operate; strict enough to protect technical quality

Overall assessment

Component	Weight
Daily GitHub work + classroom labs	15%
Holiday Mini Project 1	15%
Holiday Mini Project 2	20%
Kubernetes + GitOps implementation	15%
Observability + DevSecOps	10%
Oct 10 Final Production Challenge	20%
Viva / troubleshooting ability	5%

Trainer checkpoints

- Do not award full marks merely because the application is running. Ask: Why this design? How can it fail? How will you detect failure? How will you recover?
- Check a rotating sample of repositories by fresh clone. README-driven reproducibility is a core criterion.
- Use short daily viva questions to distinguish genuine understanding from copied configuration.
- Failure injection is mandatory on Kubernetes, observability and final challenge days.
- Prioritize one standard stack and one common cloud target. Avoid spending time on tool-brand comparisons.
- Students who finish early should improve automation, tests, observability, reliability and documentation rather than add unrelated technologies.

Scope intentionally excluded from this 12-day program

- Deep Jenkins + GitLab CI + GitHub Actions simultaneously - one primary CI platform is enough.
- Dual-cloud implementation in class - select AWS or Azure as the common target.
- Deep service mesh, Kafka, ELK/EFK, advanced policy engines and multiple Kubernetes distributions.
- GenAI/MLOps or another unrelated subject running in parallel with this 72-hour track.
- Cloud fundamentals already covered by students in prior AWS/Azure learning.

Depth over tool count: the objective is a student who can productionize and operate one application end-to-end, not a student who has merely seen many DevOps tools.

Final Student Submission Checklist

A single-page verification sheet for Oct 10

Area	Minimum evidence	Verified
Source & Git	Meaningful history; issues/branches/PRs; final release tag; no secrets	☐
Application	Python API works; tests pass; health/readiness endpoints available	☐
Container	Optimized Dockerfile; Compose stack; PostgreSQL/Redis integration	☐
CI/CD	Automated lint/test/build; versioned image; deployment traceability; rollback path	☐
Infrastructure	Terraform configuration is formatted/validated; variables/outputs; no credentials committed	☐
Kubernetes	Deployment/Service/config/probes/resources; tested rollout/rollback	☐
Helm & GitOps	Lintable chart; environment values; Argo CD/Git-driven deployment evidence	☐
Observability	Useful dashboard/metrics; failure investigation evidence; SLO statement	☐
DevSecOps	Repository/image/config scans; key findings remediated; secure secrets practice	☐
Operations	Runbook, architecture, troubleshooting and incident report	☐

Expected graduate statement

“Give me a Python backend repository. I can test it, containerize it, provision its infrastructure, deploy it through CI/CD and Kubernetes, secure and observe it, scale it, diagnose failures and recover it.”

END OF PROGRAM PLAN